

CHƯƠNG 13. TẢI VÀ TIỀN XỬ LÝ DỮ LIỆU VỚI TENSORFLOW

Trong Chương 2, bạn đã thấy rằng tải và tiền xử lý dữ liệu là một phần quan trọng của bất kỳ dự án học máy nào. Bạn đã sử dụng Pandas để tải và khám phá tập dữ liệu nhà ở California (đã sửa đổi) — được lưu trữ trong tệp CSV — và bạn đã áp dụng các bộ biến đổi của Scikit-Learn để tiền xử lý. Các công cụ này khá tiện lợi, và bạn có thể sẽ thường xuyên sử dụng chúng, đặc biệt là khi khám phá và thử nghiệm dữ liệu.

Tuy nhiên, khi huấn luyện các mô hình TensorFlow trên các tập dữ liệu lớn, bạn có thể thích sử dụng API tải và tiền xử lý dữ liệu của riêng TensorFlow, được gọi là `tf.data`. Nó có khả năng tải và tiền xử lý dữ liệu cực kỳ hiệu quả, đọc từ nhiều tệp song song bằng đa luồng và hàng đợi, xáo trộn và nhóm mẫu, v.v. Hơn nữa, nó có thể làm tất cả những điều này ngay lập tức — nó tải và tiền xử lý batch dữ liệu tiếp theo trên nhiều lõi CPU, trong khi GPU hoặc TPU của bạn đang bận huấn luyện batch dữ liệu hiện tại.

API `tf.data` cho phép bạn xử lý các tập dữ liệu không vừa trong bộ nhớ, và nó cho phép bạn tận dụng tối đa tài nguyên phần cứng của mình, từ đó tăng tốc quá trình huấn luyện. Ngoài ra, API `tf.data` có thể đọc từ các tệp văn bản (chẳng hạn như tệp CSV), các tệp nhị phân có bản ghi kích thước cố định, và các tệp nhị phân sử dụng định dạng TFRecord của TensorFlow, định dạng này hỗ trợ các bản ghi có kích thước thay đổi. TFRecord là một định dạng nhị phân linh hoạt và hiệu quả thường chứa các bộ đệm giao thức (protocol buffers - một định dạng nhị phân mã nguồn mở). API `tf.data` cũng có hỗ trợ đọc từ các cơ sở dữ liệu SQL. Hơn nữa, nhiều tiện ích mở rộng mã nguồn mở có sẵn để đọc từ tất cả các loại nguồn dữ liệu, chẳng hạn như dịch vụ BigQuery của Google (xem <https://tensorflow.org/io>).

Keras cũng đi kèm với các lớp tiền xử lý mạnh mẽ nhưng dễ sử dụng có thể được nhúng vào các mô hình của bạn: bằng cách này, khi bạn triển khai một mô hình vào sản xuất, nó sẽ có thể nhận dữ liệu thô trực tiếp, mà không cần bạn phải thêm bất kỳ mã tiền xử lý bổ sung nào. Điều này loại bỏ nguy cơ không khớp giữa mã tiền xử lý được sử dụng trong quá trình huấn luyện và mã tiền xử lý được sử dụng trong sản xuất, điều này có thể gây ra sai lệch huấn luyện/phục vụ. Và nếu bạn triển khai mô hình của mình trong nhiều ứng dụng được mã hóa bằng các ngôn ngữ lập trình khác nhau, bạn sẽ không phải triển khai lại cùng một mã tiền xử lý nhiều lần, điều này cũng giảm nguy cơ không khớp.

Như bạn sẽ thấy, cả hai API có thể được sử dụng cùng nhau — ví dụ, để hưởng lợi từ việc tải dữ liệu hiệu quả do `tf.data` cung cấp và sự tiện lợi của các lớp tiền xử lý Keras.

Trong chương này, chúng ta sẽ đề cập đến API `tf.data` và định dạng TFRecord trước. Sau đó, chúng ta sẽ khám phá các lớp tiền xử lý Keras và cách sử dụng chúng với API `tf.data`. Cuối cùng, chúng ta sẽ xem xét nhanh một vài thư viện liên quan mà bạn có thể thấy hữu ích để tải và tiền xử lý dữ liệu, chẳng hạn như TensorFlow Datasets và TensorFlow Hub. Vậy, hãy bắt đầu!

API `tf.data`

Toàn bộ API `tf.data` xoay quanh khái niệm về một `tf.data.Dataset`: điều này đại diện cho một chuỗi các mục dữ liệu. Thông thường bạn sẽ sử dụng các tập dữ liệu đọc dần dữ liệu từ đĩa, nhưng để đơn giản, hãy tạo một tập dữ liệu từ một tensor dữ liệu đơn giản bằng cách sử dụng `tf.data.Dataset.from_tensor_slices()`:

```
>>> import tensorflow as tf

>>> X = tf.range(10) # bất kỳ tensor dữ liệu nào

>>> dataset = tf.data.Dataset.from_tensor_slices(X)

>>> dataset
<TensorSliceDataset shapes:(), types: tf.int32>
```

Hàm `from_tensor_slices()` nhận một tensor và tạo một `tf.data.Dataset` mà các phần tử của nó là tất cả các lát cắt của `X` dọc theo chiều đầu tiên, vì vậy tập dữ liệu này chứa 10 mục: các tensor 0, 1, 2, ..., 9. Trong trường hợp này, chúng ta sẽ thu được cùng tập dữ liệu nếu chúng ta đã sử dụng `tf.data.Dataset.range(10)` (ngoại trừ các phần tử sẽ là số nguyên 64-bit thay vì số nguyên 32-bit).

Bạn có thể đơn giản lặp qua các mục của một tập dữ liệu như thế này:

```
>>> for item in dataset:
...     print(item)
...
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
[...]
tf.Tensor(9, shape=(), dtype=int32)
```

Một tập dữ liệu cũng có thể chứa các bộ (tuple) tensor, hoặc các từ điển cặp tên/tensor, hoặc thậm chí các bộ và từ điển lồng nhau của các tensor. Khi lát cắt một bộ, một từ điển, hoặc một cấu trúc lồng nhau, tập dữ liệu sẽ chỉ lát cắt các tensor mà nó chứa, trong khi vẫn giữ nguyên cấu trúc bộ/từ điển. Ví dụ:

```
>>> X_nested = {"a": ([1, 2, 3], [4, 5, 6]), "b": [7, 8, 9]}

>>> dataset = tf.data.Dataset.from_tensor_slices(X_nested)

>>> for item in dataset:
...     print(item)
...
{'a': (<tf.Tensor: [...] = 1>, <tf.Tensor: [...] = 4>), 'b':
<tf.Tensor: [...] = 7>}
{'a': (<tf.Tensor: [...] = 2>, <tf.Tensor: [...] = 5>), 'b':
<tf.Tensor: [...] = 8>}
```

```
{'a': (<tf.Tensor: [...] = 3>, <tf.Tensor: [...] = 6>), 'b':  
<tf.Tensor: [...] = 9>}
```

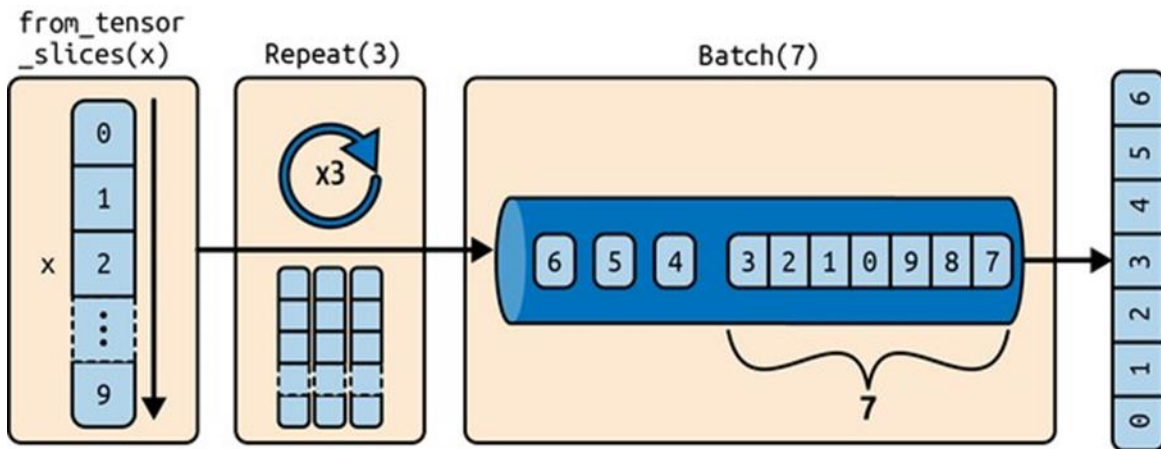
Xếp chuỗi các phép biến đổi

Khi bạn có một tập dữ liệu, bạn có thể áp dụng tất cả các loại phép biến đổi cho nó bằng cách gọi các phương thức biến đổi của nó. Mỗi phương thức trả về một tập dữ liệu mới, vì vậy bạn có thể xếp chuỗi các phép biến đổi như thế này (chuỗi này được minh họa trong Hình 13-1):

```
>>> dataset = tf.data.Dataset.from_tensor_slices(tf.range(10))  
  
>>> dataset = dataset.repeat(3).batch(7)  
  
>>> for item in dataset:  
...     print(item)  
...  
  
tf.Tensor([0 1 2 3 4 5 6], shape=(7,), dtype=int32)  
tf.Tensor([7 8 9 0 1 2 3], shape=(7,), dtype=int32)  
tf.Tensor([4 5 6 7 8 9 0], shape=(7,), dtype=int32)  
tf.Tensor([1 2 3 4 5 6 7], shape=(7,), dtype=int32)  
tf.Tensor([8 9], shape=(2,), dtype=int32)
```

Trong ví dụ này, chúng ta đầu tiên gọi phương thức `repeat()` trên tập dữ liệu gốc, và nó trả về một tập dữ liệu mới lặp lại các mục của tập dữ liệu gốc ba lần. Tất nhiên, điều này sẽ không sao chép tất cả dữ liệu vào bộ nhớ ba lần! Nếu bạn gọi phương thức này mà không có đối số, tập dữ liệu mới sẽ lặp lại tập dữ liệu nguồn mãi mãi, vì vậy mã lặp qua tập dữ liệu sẽ phải quyết định khi nào dừng lại.

Sau đó, chúng ta gọi phương thức `batch()` trên tập dữ liệu mới này, và một lần nữa điều này tạo ra một tập dữ liệu mới. Cái này sẽ nhóm các mục của tập dữ liệu trước đó thành các batch gồm bảy mục.



Hình 13-1. Xếp chuỗi các phép biến đổi tập dữ liệu

Cuối cùng, chúng ta lặp qua các mục của tập dữ liệu cuối cùng này. Phương thức `batch()` phải xuất ra một batch cuối cùng có kích thước hai thay vì bảy, nhưng bạn có thể gọi `batch()` với `drop_remainder=True` nếu bạn muốn nó bỏ batch cuối cùng này, sao cho tất cả các batch có cùng kích thước chính xác.

Bạn cũng có thể biến đổi các mục bằng cách gọi phương thức `map()`. Ví dụ, đoạn mã này tạo một tập dữ liệu mới với tất cả các batch được nhân với hai:

```
>>> dataset = dataset.map(lambda x: x * 2) # x là một batch

>>> for item in dataset:
...     print(item)
...
tf.Tensor([ 0  2  4  6  8 10 12], shape=(7,), dtype=int32)
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
[...]
```

Phương thức `map()` này là phương thức bạn sẽ gọi để áp dụng bất kỳ tiền xử lý nào cho dữ liệu của mình. Đôi khi điều này sẽ bao gồm các phép tính có thể khá chuyên sâu, chẳng hạn như định hình lại hoặc xoay hình ảnh, vì vậy bạn thường sẽ muốn tạo nhiều luồng để tăng tốc mọi thứ. Điều này có thể được thực hiện bằng cách đặt đối số `num_parallel_calls` thành số luồng để chạy, hoặc thành `tf.data.AUTOTUNE`. Lưu ý rằng hàm bạn truyền cho phương thức `map()` phải có thể chuyển đổi thành hàm TF (xem Chương 12).

Cũng có thể chỉ đơn giản là lọc tập dữ liệu bằng cách sử dụng phương thức `filter()`. Ví dụ, đoạn mã này tạo một tập dữ liệu chỉ chứa các batch có tổng lớn hơn 50:

```
>>> dataset = dataset.filter(lambda x: tf.reduce_sum(x) > 50)

>>> for item in dataset:
...     print(item)
...
[...]
```

```
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)
tf.Tensor([ 2  4  6  8 10 12 14], shape=(7,), dtype=int32)
```

Bạn thường sẽ muốn xem xét chỉ một vài mục từ một tập dữ liệu. Bạn có thể sử dụng phương thức `take()` cho điều đó:

```
>>> for item in dataset.take(2):
...     print(item)
...
tf.Tensor([14 16 18  0  2  4  6], shape=(7,), dtype=int32)
tf.Tensor([ 8 10 12 14 16 18  0], shape=(7,), dtype=int32)
```

Xáo trộn dữ liệu (Shuffling the Data)

Như chúng ta đã thảo luận trong Chương 4, giảm độ dốc hoạt động tốt nhất khi các thể hiện trong tập huấn luyện độc lập và phân phối giống hệt nhau (IID). Một cách đơn giản để đảm bảo điều này là xáo trộn các thể hiện, bằng cách sử dụng phương thức `shuffle()`. Nó sẽ tạo ra một tập dữ liệu mới bắt đầu bằng cách điền vào một bộ đệm với các mục đầu tiên của tập dữ liệu nguồn. Sau đó, bất cứ khi nào nó được yêu cầu một mục, nó sẽ kéo ngẫu nhiên một mục ra khỏi bộ đệm và thay thế nó bằng một mục mới từ tập dữ liệu nguồn, cho đến khi nó đã lặp hoàn toàn qua tập dữ liệu nguồn. Tại thời điểm này, nó sẽ tiếp tục kéo các mục ngẫu nhiên ra khỏi bộ đệm cho đến khi bộ đệm trống. Bạn phải chỉ định kích thước bộ đệm, và điều quan trọng là phải làm cho nó đủ lớn, nếu không việc xáo trộn sẽ không hiệu quả lắm. Chỉ cần không vượt quá lượng RAM bạn có, mặc dù ngay cả khi bạn có nhiều RAM, không cần vượt quá kích thước của tập dữ liệu. Bạn có thể cung cấp một hạt giống ngẫu nhiên nếu bạn muốn cùng một thứ tự ngẫu nhiên mỗi khi bạn chạy chương trình của mình. Ví dụ, đoạn mã sau tạo và hiển thị một tập dữ liệu chứa các số nguyên từ 0 đến 9, lặp lại hai lần, được xáo trộn bằng cách sử dụng bộ đệm có kích thước 4 và hạt giống ngẫu nhiên 42, và được nhóm với kích thước batch là 7:

```
>>> dataset = tf.data.Dataset.range(10).repeat(2)

>>> dataset = dataset.shuffle(buffer_size=4, seed=42).batch(7)

>>> for item in dataset:
...     print(item)
...

tf.Tensor([3 0 1 6 2 5 7], shape=(7,), dtype=int64)
tf.Tensor([8 4 1 9 4 2 3], shape=(7,), dtype=int64)
```

```
tf.Tensor([7 5 0 8 9 6], shape=(6,), dtype=int64)
```

Đối với một tập dữ liệu lớn không vừa trong bộ nhớ, cách tiếp cận bộ đệm xáo trộn đơn giản này có thể không đủ, vì bộ đệm sẽ nhỏ so với tập dữ liệu. Một giải pháp là xáo trộn dữ liệu nguồn đó (ví dụ, trên Linux bạn có thể xáo trộn các tệp văn bản bằng lệnh `shuf`). Điều này chắc chắn sẽ cải thiện việc xáo trộn rất nhiều! Ngay cả khi dữ liệu nguồn được xáo trộn, bạn thường sẽ muốn xáo trộn nó nhiều hơn, nếu không thứ tự tương tự sẽ được lặp lại ở mỗi epoch, và mô hình có thể bị sai lệch (ví dụ: do một số mẫu giả có mặt ngẫu nhiên trong thứ tự dữ liệu nguồn). Để xáo trộn các thể hiện nhiều hơn, một cách tiếp cận phổ biến là chia dữ liệu nguồn thành nhiều tệp, sau đó đọc chúng theo thứ tự ngẫu nhiên trong quá trình huấn luyện. Tuy nhiên, các thể hiện nằm trong cùng một tệp vẫn sẽ kết thúc gần nhau. Để tránh điều này, bạn có thể chọn nhiều tệp ngẫu nhiên và đọc chúng đồng thời, xen kẽ các bản ghi của chúng. Sau đó, trên hết bạn có thể thêm một bộ đệm xáo trộn bằng phương thức

`shuffle()`. Nếu điều này nghe có vẻ là rất nhiều công việc, đừng lo lắng: API

`tf.data` làm cho tất cả điều này có thể chỉ trong vài dòng mã. Hãy cùng xem cách bạn có thể làm điều này.

Xen kẽ các dòng từ nhiều tệp (Interleaving Lines from Multiple Files)

Đầu tiên, giả sử bạn đã tải tập dữ liệu nhà ở California, xáo trộn nó (trừ khi nó đã được xáo trộn), và chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra. Sau đó, bạn chia mỗi tập thành nhiều tệp CSV, mỗi tệp trông như thế này (mỗi hàng chứa tám đặc trưng đầu vào cộng với giá trị nhà trung bình mục tiêu):

```
MedInc,HouseAge,AveRooms,AveBedrms,Popul...,AveOccup,Lat...,Long...,MedianHouseValue
3.5214,15.0,3.050,1.107,1447.0,1.606,37.63,-122.43,1.442
5.3275,5.0,6.490,0.991,3464.0,3.443,33.69,-117.39,1.687
3.1,29.0,7.542,1.592,1328.0,2.251,38.44,-122.98,1.621
[...]
```

Giả sử `train_filepaths` chứa danh sách các đường dẫn tệp huấn luyện (và bạn cũng có `valid_filepaths` và `test_filepaths`):

```
>>> train_filepaths
['datasets/housing/my_train_00.csv', 'datasets/housing/my_train_01.csv', ...]
```

Ngoài ra, bạn có thể sử dụng các mẫu tệp; ví dụ,

`train_filepaths = "datasets/housing/my_train_*.csv"`. Bây giờ hãy tạo một tập dữ liệu chỉ chứa các đường dẫn tệp này:

```
filepath_dataset = tf.data.Dataset.list_files(train_filepaths,seed=42)
```

Theo mặc định, hàm

`list_files()` trả về một tập dữ liệu xáo trộn các đường dẫn tệp. Nói chung đây là một điều tốt, nhưng bạn có thể đặt

`shuffle=False` nếu bạn không muốn điều đó vì một lý do nào đó.

Tiếp theo, bạn có thể gọi phương thức

`interleave()` để đọc từ năm tệp cùng một lúc và xen kẽ các dòng của chúng. Bạn cũng có thể bỏ qua dòng đầu tiên của mỗi tệp — đó là hàng tiêu đề — bằng phương thức

`skip()`:

```
n_readers = 5
dataset = filepath_dataset.interleave(
    lambda filepath:tf.data.TextLineDataset(filepath).skip(1), cycle_length=n_readers)
```

Phương thức

`interleave()` sẽ tạo một tập dữ liệu sẽ lấy năm đường dẫn tệp từ `filepath_dataset`, và đối với mỗi đường dẫn, nó sẽ gọi hàm bạn đã cung cấp cho nó (một `lambda` trong ví dụ này) để tạo một tập dữ liệu mới (trong trường hợp này là `TextLineDataset`). Để rõ ràng, ở giai đoạn này sẽ có tổng cộng bảy tập dữ liệu: tập dữ liệu đường dẫn tệp, tập dữ liệu xen kẽ và năm `TextLineDataset` được tạo nội bộ bởi tập dữ liệu xen kẽ. Khi bạn lặp qua tập dữ liệu xen kẽ, nó sẽ luân phiên qua năm `TextLineDataset` này, đọc từng dòng một từ mỗi tập dữ liệu cho đến khi tất cả các tập dữ liệu hết mục. Sau đó, nó sẽ lấy năm đường dẫn tệp tiếp theo từ `filepath_dataset` và xen kẽ chúng theo cùng một cách, v.v. cho đến khi hết đường dẫn tệp. Để việc xen kẽ hoạt động tốt nhất, nên có các tệp có độ dài giống hệt nhau; nếu không, phần cuối của tệp dài nhất sẽ không được xen kẽ.

Theo mặc định, `interleave()` không sử dụng song song hóa; nó chỉ đọc từng dòng một từ mỗi tệp, tuần tự. Nếu bạn muốn nó thực sự đọc tệp song song, bạn có thể đặt đối số `num_parallel_calls` của phương thức `interleave()` thành số luồng bạn muốn (nhớ rằng phương thức `map()` cũng có đối số này). Bạn thậm chí có thể đặt nó thành `tf.data.AUTOTUNE` để TensorFlow chọn số luồng phù hợp một cách linh hoạt dựa trên CPU có sẵn. Hãy xem tập dữ liệu hiện chứa gì:

```
>>> for line in dataset.take(5):
...     print(line)
...
tf.Tensor(b'4.5909,16.0,[...],33.63,-117.71,2.418', shape=(),
dtype=string)
tf.Tensor(b'2.4792,24.0,[...],34.18,-118.38,2.0', shape=(),
dtype=string)
tf.Tensor(b'4.2708,45.0,[...],37.48,-122.19,2.67', shape=(),
dtype=string)
tf.Tensor(b'2.1856,41.0,[...],32.76,-117.12,1.205', shape=(),
```

```
dtype=string)
tf.Tensor(b'4.1812,52.0,[...],33.73,-118.31,3.215', shape=(),
dtype=string)
```

Đây là năm hàng đầu tiên (bỏ qua hàng tiêu đề) của năm tệp CSV, được chọn ngẫu nhiên. Trông tốt!

Tiền xử lý dữ liệu (Preprocessing the Data)

Bây giờ chúng ta có một tập dữ liệu nhà ở trả về mỗi thể hiện dưới dạng một tensor chứa một chuỗi byte, chúng ta cần thực hiện một chút tiền xử lý, bao gồm phân tích cú pháp các chuỗi và chia tỷ lệ dữ liệu.

Hãy triển khai một vài hàm tùy chỉnh sẽ thực hiện tiền xử lý này:

```
# Giả định X_mean và X_std đã được định nghĩa là tensor hoặc mảng NumPy
# Ví dụ:
# X_mean = tf.constant([1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0], dtype=tf.float32)
# X_std = tf.constant([0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2], dtype=tf.float32)

X_mean, X_std = [...] # trung bình và tỷ lệ của mỗi đặc trưng trong tập huấn luyện
n_inputs = 8 [cite: 1]

def parse_csv_line(line):
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    fields = tf.io.decode_csv(line, record_defaults=defs)
    return tf.stack(fields[:-1]), tf.stack(fields[-1:])

def preprocess(line):
    x, y = parse_csv_line(line)
    return (x - X_mean) / X_std, y
```

Hãy cùng xem đoạn mã này:

- Đầu tiên, mã giả định rằng chúng ta đã tính toán trước trung bình và độ lệch chuẩn của mỗi đặc trưng trong tập huấn luyện. `X_mean` và `X_std` chỉ là các tensor 1D (hoặc mảng NumPy) chứa tám số thập phân, một cho mỗi đặc trưng đầu vào. Điều này có thể được thực hiện bằng cách sử dụng `StandardScaler` của `Scikit-Learn` trên một mẫu ngẫu nhiên đủ lớn của tập dữ liệu. Sau này trong chương này, chúng ta sẽ sử dụng một lớp tiền xử lý Keras thay thế.
- Hàm `parse_csv_line()` nhận một dòng CSV và phân tích cú pháp nó. Để giúp việc đó, nó sử dụng hàm `tf.io.decode_csv()`, hàm này nhận hai đối số: đối số đầu tiên là dòng cần phân tích cú pháp, và đối số thứ hai là một mảng chứa giá trị mặc định cho mỗi cột trong tệp CSV. Mảng này (`defs`) không chỉ cho TensorFlow biết giá trị mặc định cho mỗi cột, mà còn cả số lượng cột và kiểu của chúng. Trong ví dụ này, chúng

ta bảo nó rằng tất cả các cột đặc trưng đều là số thập phân và các giá trị bị thiếu nên mặc định là 0, nhưng chúng ta cung cấp một mảng trống kiểu `tf.float32` làm giá trị mặc định cho cột cuối cùng (mục tiêu): mảng này cho TensorFlow biết rằng cột này chứa số thập phân, nhưng không có giá trị mặc định, vì vậy nó sẽ đưa ra một ngoại lệ nếu nó gặp một giá trị bị thiếu.

- Hàm `tf.io.decode_csv()` trả về một danh sách các tensor vô hướng (một cho mỗi cột), nhưng chúng ta cần trả về một mảng tensor 1D. Vì vậy, chúng ta gọi `tf.stack()` trên tất cả các tensor ngoại trừ tensor cuối cùng (mục tiêu): điều này sẽ xếp chồng các tensor này thành một mảng 1D. Sau đó, chúng ta làm tương tự cho giá trị mục tiêu: điều này làm cho nó trở thành một mảng tensor 1D với một giá trị duy nhất, thay vì một tensor vô hướng. Hàm `tf.io.decode_csv()` đã hoàn thành, vì vậy nó trả về các đặc trưng đầu vào và mục tiêu.
- Cuối cùng, hàm `preprocess()` tùy chỉnh chỉ gọi hàm `parse_csv_line()`, chia tỷ lệ các đặc trưng đầu vào bằng cách trừ trung bình đặc trưng và sau đó chia cho độ lệch chuẩn đặc trưng, và trả về một bộ (tuple) chứa các đặc trưng đã chia tỷ lệ và mục tiêu.

Hãy kiểm tra hàm tiền xử lý này:

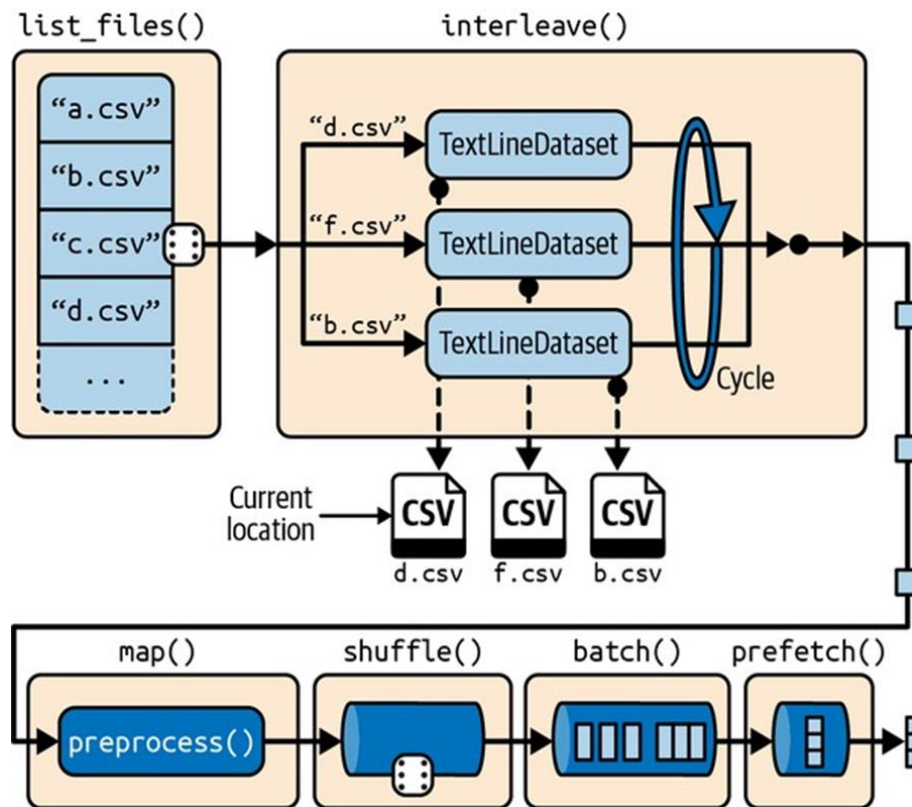
```
>>> preprocess(b'4.2083,44.0,5.3232,0.9171,846.0,2.3370,37.47,-122.2,2.782')
(<tf.Tensor: shape=(8,), dtype=float32, numpy=
array([ 0.16579159,  1.216324 , -0.05204564, -0.39215982,
        -0.5277444 , -0.2633488 ,  0.8543046 , -1.3072058 ], dtype=float32)>,
 <tf.Tensor: shape=(1,), dtype=float32, numpy=array([2.782], dtype=float32)>)
```

Trông tốt! Hàm `preprocess()` có thể chuyển đổi một thể hiện từ một chuỗi byte thành một tensor được chia tỷ lệ đẹp mắt, với nhãn tương ứng của nó. Bây giờ chúng ta có thể sử dụng phương thức `map()` của tập dữ liệu để áp dụng hàm `preprocess()` cho mỗi mẫu trong tập dữ liệu.

Để mã dễ tái sử dụng hơn, hãy gom tất cả những gì chúng ta đã thảo luận cho đến nay vào một hàm trợ giúp khác; nó sẽ tạo và trả về một tập dữ liệu sẽ tải hiệu quả dữ liệu nhà ở California từ nhiều tệp CSV, tiền xử lý, xáo trộn và nhóm nó (xem Hình 13-2):

```
def csv_reader_dataset(filepaths, n_readers=5, n_read_threads=None,
                        n_parse_threads=5, shuffle_buffer_size=10_000, seed=42
,
                        batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths, seed=seed)
    dataset = dataset.interleave(
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers, num_parallel_calls=n_read_threads)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
    dataset = dataset.shuffle(shuffle_buffer_size, seed=seed)
    return dataset.batch(batch_size).prefetch(1)
```

Lưu ý rằng chúng ta sử dụng phương thức `prefetch()` ở dòng cuối cùng. Điều này quan trọng đối với hiệu suất, như bạn sẽ thấy ngay bây giờ.

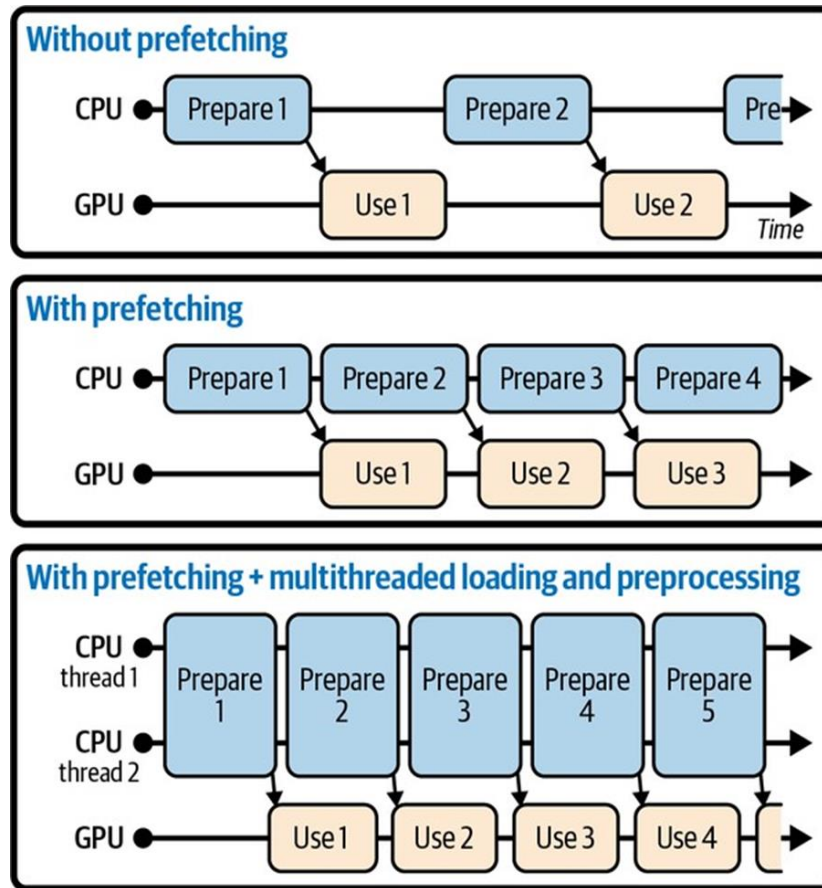


Hình 13-2. Tải và tiền xử lý dữ liệu từ nhiều tệp CSV

Prefetching

Bằng cách gọi `prefetch(1)` ở cuối hàm `csv_reader_dataset()` tùy chỉnh, chúng ta đang tạo một tập dữ liệu sẽ cố gắng hết sức để luôn đi trước một batch. Nói cách khác, trong khi thuật toán huấn luyện của chúng ta đang làm việc trên một batch, tập dữ liệu sẽ đã làm việc song song để chuẩn bị batch tiếp theo (ví dụ: đọc dữ liệu từ đĩa và tiền xử lý nó). Điều này có thể cải thiện hiệu suất đáng kể, như được minh họa trong Hình 13-3.

Nếu chúng ta cũng đảm bảo rằng việc tải và tiền xử lý được đa luồng (bằng cách đặt `num_parallel_calls` khi gọi `interleave()` và `map()`), chúng ta có thể khai thác nhiều lõi CPU và hy vọng làm cho việc chuẩn bị một batch dữ liệu ngắn hơn so với chạy một bước huấn luyện trên GPU: bằng cách này, GPU sẽ được sử dụng gần như 100% (ngoại trừ thời gian truyền dữ liệu từ CPU sang GPU), và quá trình huấn luyện sẽ chạy nhanh hơn nhiều.



Hình 13-3. Với prefetching, CPU và GPU hoạt động song song: khi GPU hoạt động trên một batch, CPU hoạt động trên batch tiếp theo

Nếu tập dữ liệu đủ nhỏ để vừa trong bộ nhớ, bạn có thể tăng tốc đáng kể quá trình huấn luyện bằng cách sử dụng phương thức `cache()` của tập dữ liệu để lưu nội dung của nó vào RAM. Bạn nên thực hiện điều này sau khi tải và tiền xử lý dữ liệu, nhưng trước khi xáo trộn, lặp lại, nhóm batch và prefetching. Bằng cách này, mỗi thể hiện sẽ chỉ được đọc và tiền xử lý một lần (thay vì một lần mỗi epoch), nhưng dữ liệu vẫn sẽ được xáo trộn khác nhau ở mỗi epoch, và batch tiếp theo vẫn sẽ được chuẩn bị trước.

Bây giờ bạn đã học cách xây dựng các pipeline đầu vào hiệu quả để tải và tiền xử lý dữ liệu từ nhiều tệp văn bản. Chúng ta đã thảo luận về các phương thức tập dữ liệu phổ biến nhất, nhưng còn một vài phương thức khác mà bạn có thể muốn xem xét, chẳng hạn như `concatenate()`, `zip()`, `window()`, `reduce()`, `shard()`, `flat_map()`, `apply()`, `unbatch()`, và `padded_batch()`. Cũng có một vài phương thức lớp khác, chẳng hạn như `from_generator()` và `from_tensors()`, cái này tạo một tập dữ liệu mới từ một trình tạo Python hoặc một danh sách các tensor, tương ứng. Vui lòng kiểm tra tài liệu API để biết thêm chi tiết. Cũng lưu ý rằng có các tính năng thử nghiệm có sẵn trong `tf.data.experimental`, nhiều tính năng trong số đó có thể sẽ được đưa vào API cốt lõi trong các bản phát hành trong tương lai (ví dụ, hãy xem lớp `CsvDataset`, cũng như phương thức `make_csv_dataset()`, cái này đảm nhiệm việc suy luận kiểu của mỗi cột).

Sử dụng Dataset với Keras

Bây giờ chúng ta có thể sử dụng hàm `csv_reader_dataset()` đã viết trước đó để tạo một tập dữ liệu cho tập huấn luyện, tập kiểm định (validation) và tập kiểm tra (test). Tập huấn luyện sẽ được xáo trộn ở mỗi epoch (lưu ý rằng tập kiểm định và tập kiểm tra cũng sẽ được xáo trộn, mặc dù chúng ta không thực sự cần điều đó):

```
train_set = csv_reader_dataset(train_filepaths)
valid_set = csv_reader_dataset(valid_filepaths)
test_set = csv_reader_dataset(test_filepaths)
```

Bây giờ bạn có thể chỉ cần xây dựng và huấn luyện một mô hình Keras bằng các tập dữ liệu này. Khi bạn gọi phương thức `fit()` của mô hình, bạn truyền `train_set` thay vì `X_train`, `y_train`, và truyền `validation_data=valid_set` thay vì `validation_data=(X_valid, y_valid)`. Phương thức `fit()` sẽ tự động lặp lại tập dữ liệu huấn luyện một lần mỗi epoch, sử dụng một thứ tự ngẫu nhiên khác nhau ở mỗi epoch:

```
model = tf.keras.Sequential([...])
model.compile(loss="mse", optimizer="sgd")
model.fit(train_set, validation_data=valid_set, epochs=5)
```

Tương tự, bạn có thể truyền một tập dữ liệu cho các phương thức `evaluate()` và `predict()`:

```
test_mse = model.evaluate(test_set)
new_set = test_set.take(3) # giả sử chúng ta có 3 mẫu mới
y_pred = model.predict(new_set) # hoặc bạn có thể chỉ cần truyền một mảng NumPy
```

Không giống như các tập khác, `new_set` thường sẽ không chứa nhãn. Nếu có, như trong trường hợp này, Keras sẽ bỏ qua chúng. Lưu ý rằng trong tất cả các trường hợp này, bạn vẫn có thể sử dụng mảng NumPy thay vì tập dữ liệu nếu bạn thích (nhưng tất nhiên chúng cần được tải và tiền xử lý trước).

Nếu bạn muốn xây dựng vòng lặp huấn luyện tùy chỉnh của riêng mình (như đã thảo luận trong Chương 12), bạn có thể chỉ cần lặp qua tập huấn luyện một cách rất tự nhiên:

```
n_epochs = 5
for epoch in range(n_epochs):
    for X_batch, y_batch in train_set:
        [...] # thực hiện một bước hạ gradient
```

Thực tế, thậm chí có thể tạo một hàm TF (xem Chương 12) để huấn luyện mô hình trong cả một epoch. Điều này thực sự có thể tăng tốc độ huấn luyện:

```
@tf.function
def train_one_epoch(model, optimizer, loss_fn, train_set):
    for X_batch, y_batch in train_set:
        with tf.GradientTape() as tape:
            y_pred = model(X_batch)
```

```

        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
for epoch in range(n_epochs):
    print("\rEpoch {}/{}".format(epoch + 1, n_epochs), end="")
    train_one_epoch(model, optimizer, loss_fn, train_set)

```

Trong Keras, đối số `steps_per_execution` của phương thức `compile()` cho phép bạn xác định số lượng lô mà phương thức `fit()` sẽ xử lý trong mỗi lần gọi đến hàm `tf.function` mà nó sử dụng để huấn luyện. Giá trị mặc định chỉ là 1, vì vậy nếu bạn đặt nó thành 50, bạn sẽ thường thấy một sự cải thiện hiệu suất đáng kể. Tuy nhiên, các phương thức `on_batch_*`() của các callback Keras sẽ chỉ được gọi sau mỗi 50 lô.

Xin chúc mừng, bây giờ bạn đã biết cách xây dựng các đường ống đầu vào mạnh mẽ bằng cách sử dụng API `tf.data`! Tuy nhiên, cho đến nay chúng ta đã sử dụng các tệp CSV, vốn phổ biến, đơn giản và tiện lợi nhưng không thực sự hiệu quả, và không hỗ trợ tốt các cấu trúc dữ liệu lớn hoặc phức tạp (như hình ảnh hoặc âm thanh). Vì vậy, hãy xem cách sử dụng `TfRecords` thay thế.

Định dạng TFRecord

Định dạng **TFRecord** là định dạng ưa thích của TensorFlow để lưu trữ lượng lớn dữ liệu và đọc nó một cách hiệu quả. Đó là một định dạng nhị phân rất đơn giản chỉ chứa một chuỗi các bản ghi nhị phân có kích thước thay đổi (mỗi bản ghi bao gồm độ dài, tổng kiểm CRC để kiểm tra xem độ dài có bị hỏng không, sau đó là dữ liệu thực tế và cuối cùng là tổng kiểm CRC cho dữ liệu). Bạn có thể dễ dàng tạo một tệp TFRecord bằng cách sử dụng lớp `tf.io.TFRecordWriter`:

```

with tf.io.TFRecordWriter("my_data.tfrecord") as f:
    f.write(b"This is the first record")
    f.write(b"And this is the second record")

```

Và sau đó bạn có thể sử dụng `tf.data.TFRecordDataset` để đọc một hoặc nhiều tệp TFRecord:

```

filepaths = ["my_data.tfrecord"]
dataset = tf.data.TFRecordDataset(filepaths)
for item in dataset:
    print(item)

```

Điều này sẽ xuất ra:

```
tf.Tensor(b'This is the first record', shape=(), dtype=string)
tf.Tensor(b'And this is the second record', shape=(), dtype=string)
```

Tập TFRecord được nén

Đôi khi có thể hữu ích khi nén các tập TFRecord của bạn, đặc biệt nếu chúng cần được tải qua kết nối mạng. Bạn có thể tạo một tập TFRecord được nén bằng cách đặt đối số options:

```
options = tf.io.TFRecordOptions(compression_type="GZIP")
with tf.io.TFRecordWriter("my_compressed.tfrecord", options) as f:
    f.write(b"Compress, compress, compress!")
```

Khi đọc một tập TFRecord được nén, bạn cần chỉ định loại nén:

```
dataset = tf.data.TFRecordDataset(["my_compressed.tfrecord"],
                                   compression_type="GZIP")
```

Giới thiệu ngắn gọn về Protocol Buffers

Mặc dù mỗi bản ghi có thể sử dụng bất kỳ định dạng nhị phân nào bạn muốn, các tập TFRecord thường chứa các **protocol buffers** (còn gọi là **protobufs**) được tuần tự hóa. Đây là một định dạng nhị phân di động, có thể mở rộng và hiệu quả được phát triển tại Google vào năm 2001 và được mã nguồn mở vào năm 2008. Protobufs hiện được sử dụng rộng rãi, đặc biệt là trong gRPC, hệ thống gọi thủ tục từ xa của Google. Chúng được định nghĩa bằng một ngôn ngữ đơn giản trông như thế này:

```
syntax = "proto3";

message Person {
    string name = 1;
    int32 id = 2;
    repeated string email = 3;
}
```

Định nghĩa protobuf này cho biết chúng ta đang sử dụng phiên bản 3 của định dạng protobuf và nó chỉ định rằng mỗi đối tượng Person có thể (tùy chọn) có một name kiểu string, một id kiểu int32 và không hoặc nhiều trường email, mỗi trường có kiểu string. Các số 1, 2 và 3 là các mã định danh trường: chúng sẽ được sử dụng trong biểu diễn nhị phân của mỗi bản ghi. Khi bạn có một định nghĩa trong tệp .proto, bạn có thể biên dịch nó. Điều này yêu cầu protoc, trình biên dịch protobuf, để tạo các lớp truy cập bằng Python (hoặc một ngôn ngữ khác).

Để minh họa những điều cơ bản, hãy xem một ví dụ đơn giản sử dụng các lớp truy cập được tạo cho protobuf Person:

```
>>> from person_pb2 import Person # nhập lớp truy cập đã tạo

>>> person = Person(name="Al", id=123, email=["a@b.com"]) # tạo một Person
>>> print(person) # hiển thị Person
```

```

name: "Al"
id: 123
email: "a@b.com"

>>> person.name # đọc một trường
'Al'
>>> person.name = "Alice" # sửa đổi một trường
>>> person.email[0] # các trường lặp lại có thể được truy cập như mảng
'a@b.com'
>>> person.email.append("c@d.com") # thêm một địa chỉ email

>>> serialized = person.SerializeToString() # tuần tự hóa person thành chuỗi
byte
>>> serialized
b'\n\x05Alice\x10{\x1a\x07a@b.com\x1a\x07c@d.com'

>>> person2 = Person() # tạo một Person mới
>>> person2.ParseFromString(serialized) # phân tích chuỗi byte (dài 27 byte)
27
>>> person == person2 # bây giờ chúng bằng nhau
True

```

Tóm lại, chúng ta nhập lớp Person được tạo bởi protoc, tạo một thể hiện và thao tác với nó, sau đó tuần tự hóa nó bằng phương thức `SerializeToString()`. Đây là dữ liệu nhị phân sẵn sàng để được lưu hoặc truyền qua mạng. Khi đọc hoặc nhận dữ liệu nhị phân này, chúng ta có thể phân tích cú pháp nó bằng phương thức `ParseFromString()` và nhận lại một bản sao của đối tượng đã được tuần tự hóa.

Các Protobuf của TensorFlow

Protobuf chính thường được sử dụng trong tệp TFRecord là **Example protobuf**, đại diện cho một mẫu trong tập dữ liệu. Nó chứa một danh sách các đặc trưng được đặt tên, trong đó mỗi đặc trưng có thể là một danh sách các chuỗi byte, một danh sách các số thực (float), hoặc một danh sách các số nguyên (integer). Đây là định nghĩa protobuf (từ mã nguồn của TensorFlow):

```

syntax = "proto3";

message BytesList { repeated bytes value = 1; }
message FloatList { repeated float value = 1 [packed = true]; }
message Int64List { repeated int64 value = 1 [packed = true]; }

message Feature {
  oneof kind {
    BytesList bytes_list = 1;
    FloatList float_list = 2;
    Int64List int64_list = 3;
  }
}

```

```

    }
};

message Features { map<string, Feature> feature = 1; };

message Example { Features features = 1; };

```

Đây là cách bạn có thể tạo một `tf.train.Example` đại diện cho cùng một người như trước đó:

```

from tensorflow.train import BytesList, FloatList, Int64List
from tensorflow.train import Feature, Features, Example

person_example = Example(
    features=Features(
        feature={
            "name": Feature(bytes_list=BytesList(value=[b"Alice"])),
            "id": Feature(int64_list=Int64List(value=[123])),
            "emails": Feature(bytes_list=BytesList(value=[b"a@b.com", b"c@d.c
om"])))
    )))

```

Mã này hơi dài dòng, nhưng bạn có thể dễ dàng gói nó vào một hàm trợ giúp nhỏ. Bây giờ chúng ta có một `Example` protobuf, chúng ta có thể tuần tự hóa nó bằng cách gọi phương thức `SerializeToString()` của nó, sau đó ghi dữ liệu kết quả vào một tệp `TFRecord`.

Tải và Phân tích cú pháp các Example

Để tải các `Example` protobuf đã được tuần tự hóa, chúng ta sẽ lại sử dụng `tf.data.TFRecordDataset` và chúng ta sẽ phân tích cú pháp mỗi `Example` bằng `tf.io.parse_single_example()`. Nó yêu cầu ít nhất hai đối số: một tensor vô hướng chuỗi chứa dữ liệu đã tuần tự hóa và một mô tả về mỗi đặc trưng. Mô tả là một từ điển ánh xạ mỗi tên đặc trưng tới một `tf.io.FixedLenFeature` (cho biết hình dạng, kiểu và giá trị mặc định của đặc trưng) hoặc một `tf.io.VarLenFeature` (chỉ cho biết kiểu nếu độ dài của danh sách đặc trưng có thể thay đổi).

```

feature_description = {
    "name": tf.io.FixedLenFeature([], tf.string, default_value=""),
    "id": tf.io.FixedLenFeature([], tf.int64, default_value=0),
    "emails": tf.io.VarLenFeature(tf.string),
}

def parse(serialized_example):
    return tf.io.parse_single_example(serialized_example, feature_description)

dataset = tf.data.TFRecordDataset(["my_contacts.tfrecord"]).map(parse)
for parsed_example in dataset:
    print(parsed_example)

```


Các đặc trưng có độ dài cố định được phân tích cú pháp thành các tensor thông thường, nhưng các đặc trưng có độ dài thay đổi được phân tích cú pháp thành các **sparse tensors**. Bạn có thể chuyển đổi một sparse tensor thành một dense tensor bằng `tf.sparse.to_dense()`.

Thay vì phân tích cú pháp các ví dụ từng cái một, bạn có thể muốn phân tích cú pháp chúng theo lô bằng `tf.io.parse_example()`.

Cuối cùng, một `ByteList` có thể chứa bất kỳ dữ liệu nhị phân nào bạn muốn, bao gồm bất kỳ đối tượng được tuần tự hóa nào. Ví dụ, bạn có thể sử dụng `tf.io.encode_jpeg()` để mã hóa một hình ảnh và đặt dữ liệu nhị phân này vào một `ByteList`.

Xử lý Danh sách của các Danh sách bằng SequenceExample Protobuf

`SequenceExample` của TensorFlow được thiết kế cho các trường hợp sử dụng như phân loại tài liệu văn bản, nơi mỗi tài liệu có thể được biểu diễn dưới dạng danh sách các câu, và mỗi câu là một danh sách các từ. Một `SequenceExample` chứa một đối tượng `Features` cho dữ liệu ngữ cảnh và một đối tượng `FeatureLists` chứa một hoặc nhiều `FeatureList` được đặt tên. Việc xây dựng, tuần tự hóa và phân tích cú pháp một `SequenceExample` tương tự như một `Example`, nhưng bạn phải sử dụng `tf.io.parse_single_sequence_example()` hoặc `tf.io.parse_sequence_example()`.

Bây giờ bạn đã biết cách lưu trữ, tải, phân tích cú pháp và tiền xử lý dữ liệu một cách hiệu quả, đã đến lúc chuyển sự chú ý của chúng ta sang các lớp tiền xử lý của Keras.

Các Lớp Tiền xử lý của Keras

Việc chuẩn bị dữ liệu của bạn cho một mạng nơ-ron thường đòi hỏi việc chuẩn hóa các đặc trưng số, mã hóa các đặc trưng phân loại và văn bản, cắt và thay đổi kích thước hình ảnh, v.v. Có một số tùy chọn cho việc này:

- **Tiền xử lý trước:** Thực hiện khi chuẩn bị các tệp dữ liệu huấn luyện của bạn, sử dụng bất kỳ công cụ nào bạn thích.
- **Tiền xử lý tức thời với `tf.data`:** Áp dụng một hàm tiền xử lý cho mọi phần tử của một tập dữ liệu bằng phương thức `map()` của nó.
- **Lớp tiền xử lý trong mô hình:** Bao gồm các lớp tiền xử lý trực tiếp bên trong mô hình của bạn.

Phần còn lại của chương này sẽ xem xét phương pháp cuối cùng. Keras cung cấp nhiều lớp tiền xử lý mà bạn có thể đưa vào mô hình của mình.

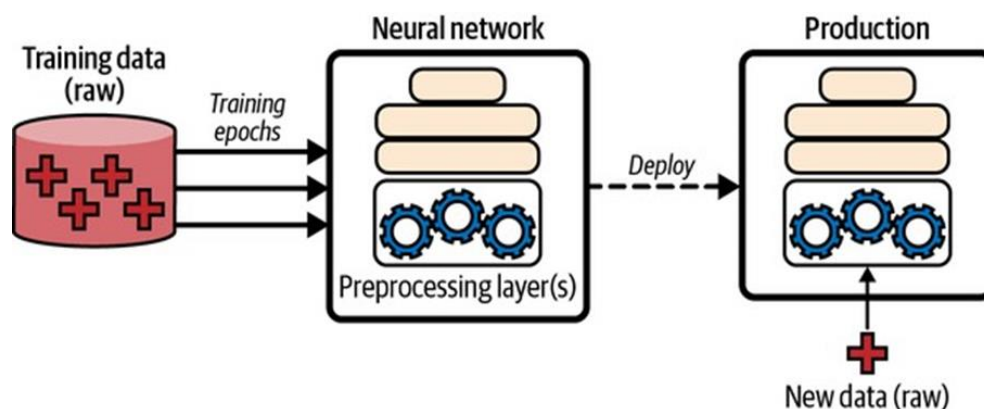
Lớp Normalization

Như chúng ta đã thấy, Keras cung cấp một lớp `Normalization` mà chúng ta có thể sử dụng để chuẩn hóa các đặc trưng đầu vào. Bạn có thể chỉ định `mean` và `variance` khi tạo lớp hoặc—

đơn giản hơn—truyền tập huấn luyện vào phương thức `adapt()` của lớp để nó tự đo lường chúng:

```
norm_layer = tf.keras.layers.Normalization()
model = tf.keras.models.Sequential([
    norm_layer,
    tf.keras.layers.Dense(1)
])
model.compile(...)
norm_layer.adapt(X_train) # tính toán giá trị trung bình và phương sai
model.fit(X_train, y_train, ...)
```

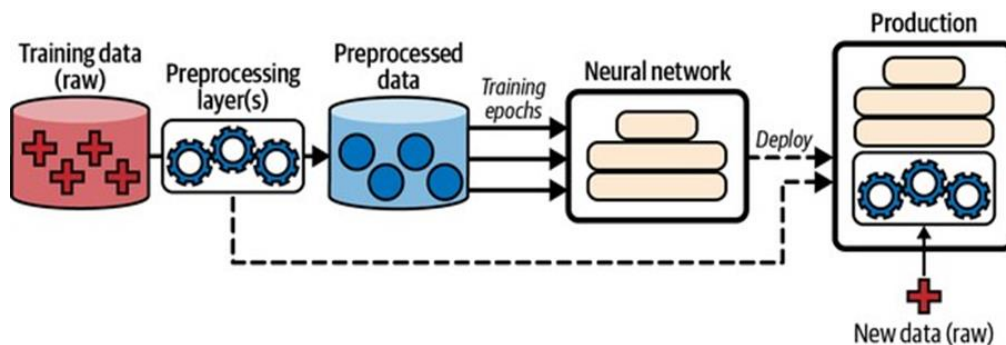
Vì chúng ta đã bao gồm lớp `Normalization` bên trong mô hình, chúng ta có thể triển khai mô hình này vào sản xuất mà không cần lo lắng về việc chuẩn hóa nữa: mô hình sẽ tự xử lý nó.



Hình 13-4. Bao gồm các lớp tiền xử lý bên trong một mô hình

Cách tiếp cận này giúp loại bỏ hoàn toàn nguy cơ **sai lệch tiền xử lý**.

Tuy nhiên, việc đưa lớp tiền xử lý trực tiếp vào mô hình sẽ làm chậm quá trình huấn luyện một chút. Để khắc phục, chúng ta có thể chuẩn hóa toàn bộ tập huấn luyện chỉ một lần trước khi huấn luyện. Sau đó, để triển khai, chúng ta tạo một mô hình mới bao bọc cả lớp `Normalization` đã được `adapt` và mô hình chúng ta vừa huấn luyện.



Hình 13-5. Tiền xử lý dữ liệu một lần trước khi huấn luyện, sau đó triển khai các lớp đó bên trong mô hình cuối cùng

Bây giờ chúng ta có được điều tốt nhất của cả hai thế giới: huấn luyện nhanh và mô hình cuối cùng có thể tự tiền xử lý đầu vào của nó.

Lớp Discretization

Lớp Discretization biến một đặc trưng số thành một đặc trưng phân loại bằng cách ánh xạ các phạm vi giá trị (gọi là **bins** - thùng) vào các danh mục. Ví dụ, mã sau đây ánh xạ một đặc trưng tuổi số thành ba loại: dưới 18, từ 18 đến 50 (không bao gồm), và 50 trở lên:

```
>>> age = tf.constant([[10.], [93.], [57.], [18.], [37.], [5.]])
>>> discretize_layer = tf.keras.layers.Discretization(bin_boundaries=[18., 50.])
>>> age_categories = discretize_layer(age)
>>> age_categories
<tf.Tensor: shape=(6, 1), dtype=int64, numpy=array([[0],[2],[2], [1],[1],[0]])>
```

Các mã định danh danh mục như thế này thường nên được mã hóa, ví dụ như sử dụng mã hóa one-hot.

Lớp CategoryEncoding

Khi chỉ có một vài danh mục, **mã hóa one-hot** thường là một lựa chọn tốt. Để làm điều này, Keras cung cấp lớp CategoryEncoding.

```
>>> onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3)
>>> onehot_layer(age_categories)
<tf.Tensor: shape=(6, 3), dtype=float32, numpy=
array([[0., 1., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.]], dtype=float32)>
```

Lớp StringLookup

Đối với các đặc trưng văn bản phân loại, bạn có thể sử dụng lớp StringLookup. Lớp này sẽ tìm tất cả các chuỗi duy nhất (từ vựng) và ánh xạ mỗi chuỗi đến một ID số nguyên. Các danh mục không xác định được ánh xạ tới 0 theo mặc định.

```
>>> cities = ["Auckland", "Paris", "Paris", "San Francisco"]
>>> str_lookup_layer = tf.keras.layers.StringLookup()
>>> str_lookup_layer.adapt(cities)
>>> str_lookup_layer(["Paris", "Auckland", "Auckland", "Montreal"])
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=array([[1], [3], [3], [0]])>
```

Nếu bạn đặt `output_mode="one_hot"`, nó sẽ xuất ra một vector one-hot cho mỗi danh mục.

Để xử lý các danh mục hiếm hoặc không xác định, bạn có thể đặt `num_oov_indices` (số lượng thùng cho từ vựng ngoài - OOV). Mỗi danh mục không xác định sẽ được ánh xạ ngẫu nhiên vào một trong các thùng OOV.

Lớp Hashing

Đối với mỗi danh mục, lớp Hashing của Keras tính toán một giá trị băm, modulo số lượng thùng. Lợi ích của lớp này là nó không cần phải được adapt, điều này đôi khi có thể hữu ích. Tuy nhiên, có thể xảy ra **xung đột băm** (hashing collision), nơi các danh mục khác nhau được ánh xạ đến cùng một ID. Vì vậy, thường thì việc sử dụng lớp StringLookup sẽ tốt hơn.

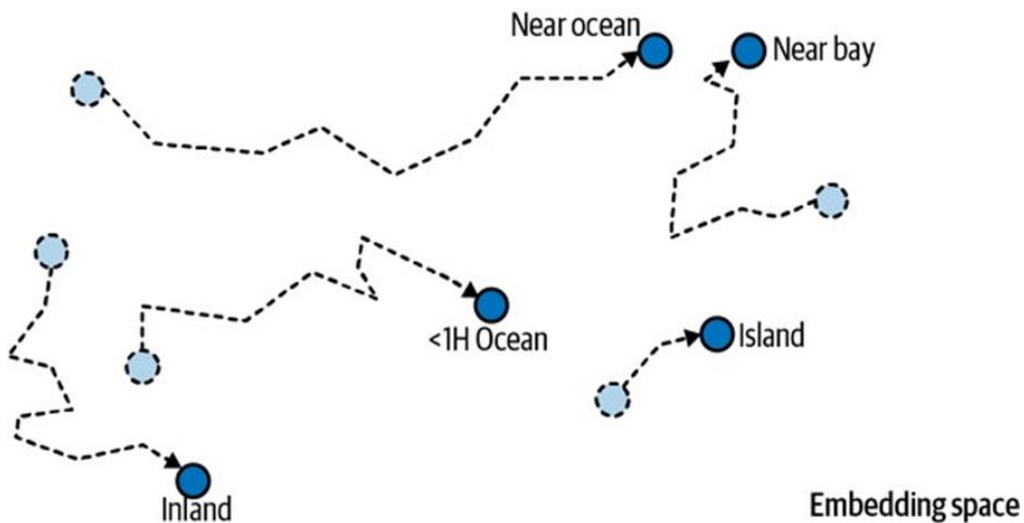
Bây giờ hãy xem một cách khác để mã hóa các danh mục: các nhúng có thể huấn luyện.

Mã hóa đặc trưng phân loại bằng Embedding

Một **embedding** là một biểu diễn dày đặc của một số dữ liệu có chiều cao hơn, chẳng hạn như một danh mục, hoặc một từ trong một từ vựng. Nếu có 50.000 danh mục có thể, thì mã hóa one-hot sẽ tạo ra một vector thưa thớt 50.000 chiều (tức là, chứa chủ yếu là số 0). Ngược lại, một embedding sẽ là một vector dày đặc tương đối nhỏ; ví dụ, chỉ với 100 chiều.

Trong học sâu, các embedding thường được khởi tạo ngẫu nhiên, và sau đó chúng được huấn luyện bằng giám độ dốc, cùng với các tham số mô hình khác. Ví dụ, danh mục “NEAR BAY” trong tập dữ liệu nhà ở California ban đầu có thể được biểu diễn bằng một vector ngẫu nhiên như $[0.131, 0.890]$, trong khi danh mục “NEAR OCEAN” có thể được biểu diễn bằng một vector ngẫu nhiên khác như $[0.631, 0.791]$. Trong ví dụ này, chúng ta sử dụng các embedding 2D, nhưng số chiều là một siêu tham số bạn có thể điều chỉnh.

Vì các embedding này có thể huấn luyện được, chúng sẽ dần dần cải thiện trong quá trình huấn luyện; và vì chúng đại diện cho các danh mục khá giống nhau trong trường hợp này, giám độ dốc chắc chắn sẽ đẩy chúng gần nhau hơn, trong khi nó sẽ có xu hướng di chuyển chúng ra xa embedding của danh mục “INLAND” (xem Hình 13-6). Thật vậy, biểu diễn càng tốt, mạng nơ-ron càng dễ đưa ra dự đoán chính xác, vì vậy quá trình huấn luyện có xu hướng làm cho các embedding trở thành biểu diễn hữu ích của các danh mục. Đây được gọi là **học biểu diễn (representation learning)** (bạn sẽ thấy các loại học biểu diễn khác trong Chương 17).



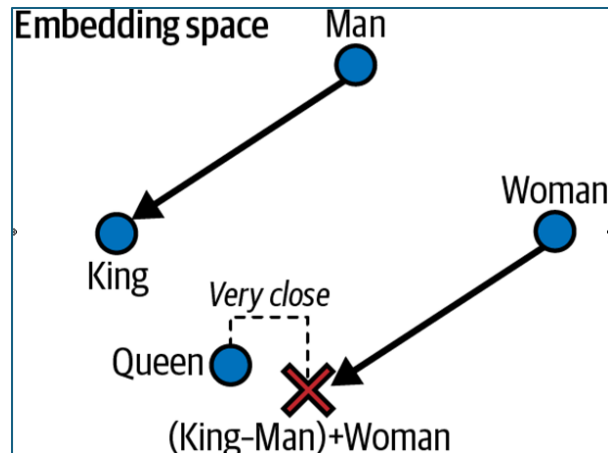
Hình 13-6. Các embedding sẽ dần dần cải thiện trong quá trình huấn luyện

WORD EMBEDDINGS (Embedding từ)

Các embedding không chỉ nói chung sẽ là biểu diễn hữu ích cho tác vụ hiện tại, mà khá thường xuyên các embedding tương tự này có thể được tái sử dụng thành công cho các tác vụ khác. Ví dụ phổ biến nhất về điều này là **embedding từ (word embeddings)** (tức là, embedding của các từ riêng lẻ): khi bạn đang làm việc với một tác vụ xử lý ngôn ngữ tự nhiên, bạn thường sẽ làm tốt hơn khi tái sử dụng các embedding từ đã được huấn luyện trước hơn là huấn luyện của riêng bạn.

Ý tưởng sử dụng vector để biểu diễn từ có từ những năm 1960, và nhiều kỹ thuật tinh vi đã được sử dụng để tạo ra các vector hữu ích, bao gồm cả việc sử dụng mạng nơ-ron. Nhưng mọi thứ thực sự bùng nổ vào năm 2013, khi Tomáš Mikolov và các nhà nghiên cứu Google khác công bố một bài báo mô tả một kỹ thuật hiệu quả để học embedding từ bằng mạng nơ-ron, vượt trội đáng kể so với các nỗ lực trước đó. Điều này cho phép họ học embedding trên một tập hợp văn bản rất lớn: họ đã huấn luyện một mạng nơ-ron để dự đoán các từ gần bất kỳ từ nào đã cho và thu được các embedding từ đáng kinh ngạc. Ví dụ, các từ đồng nghĩa có các embedding rất gần nhau, và các từ có liên quan về mặt ngữ nghĩa như France, Spain, và Italy cuối cùng được nhóm lại với nhau.

Tuy nhiên, nó không chỉ là về sự gần gũi: embedding từ cũng được tổ chức dọc theo các trục có ý nghĩa trong không gian embedding. Đây là một ví dụ nổi tiếng: nếu bạn tính toán King - Man + Woman (cộng và trừ các vector embedding của các từ này), thì kết quả sẽ rất gần với embedding của từ Queen (xem Hình 13-7). Nói cách khác, embedding từ mã hóa khái niệm giới tính! Tương tự, bạn có thể tính Madrid - Spain + France, và kết quả gần với Paris, điều này dường như cho thấy khái niệm thủ đô cũng được mã hóa trong embedding.



Hình 13-7. Embedding từ của các từ tương tự có xu hướng gần nhau, và một số trục đường như mã hóa các khái niệm có ý nghĩa

Thật không may, embedding từ đôi khi nắm bắt những thành kiến tồi tệ nhất của chúng ta. Ví dụ, mặc dù chúng học đúng rằng Man tương ứng với King như Woman tương ứng với Queen, nhưng chúng cũng dường như học rằng Man tương ứng với Doctor như Woman tương ứng với Nurse: một thành kiến phân biệt giới tính! Công bằng mà nói, ví dụ cụ thể này có lẽ đã bị phóng đại, như đã được chỉ ra trong một bài báo năm 2019 của Malvina Nissim et al. Tuy nhiên, đảm bảo công bằng trong các thuật toán học sâu là một chủ đề nghiên cứu quan trọng và tích cực.

Keras cung cấp một lớp Embedding, lớp này bao bọc một ma trận embedding: ma trận này có một hàng cho mỗi danh mục và một cột cho mỗi chiều embedding. Theo mặc định, nó được khởi tạo ngẫu nhiên. Để chuyển đổi một ID danh mục thành một embedding, lớp Embedding chỉ cần tra cứu và trả về hàng tương ứng với danh mục đó. Thế thôi! Ví dụ, hãy khởi tạo một lớp Embedding với năm hàng và các embedding 2D, và sử dụng nó để mã hóa một số danh mục:

```
>>> tf.random.set_seed(42)

>>> embedding_layer = tf.keras.layers.Embedding(input_dim=5, output_dim=2)

>>> embedding_layer(np.array([2, 4, 2]))
<tf.Tensor:
shape=(3, 2), dtype=float32, numpy= array([[ -0.04663396,  0.01846724],
      [ -0.02736737, -0.02768031],
      [ -0.04663396,  0.01846724]], dtype=float32)>
```

Như bạn có thể thấy, danh mục 2 được mã hóa (hai lần) dưới dạng vector 2D $[-0.04663396, 0.01846724]$, trong khi danh mục 4 được mã hóa là $[-0.02736737, -0.02768031]$. Vì lớp chưa được huấn luyện, các mã hóa này chỉ là ngẫu nhiên.

Nếu bạn muốn nhúng một thuộc tính văn bản phân loại, bạn có thể đơn giản nối chuỗi một lớp StringLookup và một lớp Embedding, như thế này:

```

>>> tf.random.set_seed(42)

>>> ocean_prox = ["<1H OCEAN", "INLAND", "NEAR OCEAN", "NEAR BAY", "ISLAND"]

>>> str_lookup_layer = tf.keras.layers.StringLookup()

>>> str_lookup_layer.adapt(ocean_prox)

>>> lookup_and_embed = tf.keras.Sequential([
...     str_lookup_layer,
...     tf.keras.layers.Embedding(input_dim=str_lookup_layer.vocabulary_size(
... ),
...                                output_dim=2)
... ])

...

>>> lookup_and_embed(np.array(["<1H OCEAN", "ISLAND", "<1H OCEAN"]))
<tf.Tensor:
shape=(3, 2), dtype=float32, numpy= array([[-0.01896119, 0.02223358],
      [ 0.02401174, 0.03724445],
      [-0.01896119, 0.02223358]], dtype=float32)>

```

Lưu ý rằng số hàng trong ma trận embedding cần phải bằng kích thước từ vựng (vocabulary_size): đó là tổng số danh mục, bao gồm các danh mục đã biết cộng với các thùng OOV (mặc định chỉ có một). Phương thức vocabulary_size() của lớp StringLookup tiện lợi trả về số này.

Kết hợp mọi thứ lại với nhau, bây giờ chúng ta có thể tạo một mô hình Keras có thể xử lý một đặc trưng văn bản phân loại cùng với các đặc trưng số thông thường và học một embedding cho mỗi danh mục (cũng như cho mỗi thùng OOV):

```

# Giả định X_train_num, X_train_cat, y_train, X_valid_num, X_valid_cat, y_val
id đã được tải
# Ví dụ:
# X_train_num = np.random.rand(100, 8).astype(np.float32)
# X_train_cat = np.array(["<1H OCEAN", "INLAND", "NEAR OCEAN", "NEAR B
AY"], ["ISLAND"]) * 20)
# y_train = np.random.rand(100, 1).astype(np.float32)
# X_valid_num = np.random.rand(20, 8).astype(np.float32)
# X_valid_cat = np.array(["<1H OCEAN", "INLAND"]) * 10)
# y_valid = np.random.rand(20, 1).astype(np.float32)

num_input = tf.keras.layers.Input(shape=[8], name="num")

```

```

cat_input = tf.keras.layers.Input(shape=[], dtype=tf.string, name="cat")
cat_embeddings = lookup_and_embed(cat_input) # sử dụng lookup_and_embed đã tạo ở trên
encoded_inputs = tf.keras.layers.concatenate([num_input, cat_embeddings])
outputs = tf.keras.layers.Dense(1)(encoded_inputs)
model = tf.keras.models.Model(inputs=[num_input, cat_input], outputs=[outputs])
model.compile(loss="mse", optimizer="sgd")
history = model.fit((X_train_num, X_train_cat), y_train, epochs=5,
                    validation_data=((X_valid_num, X_valid_cat),
                                    y_valid))

```

Mô hình này nhận hai đầu vào: `num_input`, chứa tám đặc trưng số cho mỗi thể hiện, cộng với `cat_input`, chứa một đầu vào văn bản phân loại duy nhất cho mỗi thể hiện. Mô hình sử dụng mô hình `lookup_and_embed` chúng ta đã tạo trước đó để mã hóa mỗi danh mục khoảng cách đại dương (ocean-proximity) thành embedding có thể huấn luyện tương ứng. Tiếp theo, nó nối các đầu vào số và các embedding bằng hàm `concatenate()` để tạo ra các đầu vào đã được mã hóa hoàn chỉnh, sẵn sàng được đưa vào mạng nơ-ron. Chúng ta có thể thêm bất kỳ loại mạng nơ-ron nào tại thời điểm này, nhưng để đơn giản, chúng ta chỉ thêm một lớp đầu ra dày đặc duy nhất, và sau đó chúng ta tạo mô hình Keras với các đầu vào và đầu ra chúng ta vừa định nghĩa. Tiếp theo, chúng ta biên dịch mô hình và huấn luyện nó, truyền cả đầu vào số và phân loại.

Như bạn đã thấy trong Chương 10, vì các lớp `Input` được đặt tên là “num” và “cat”, chúng ta cũng có thể đã truyền dữ liệu huấn luyện cho phương thức `fit()` bằng cách sử dụng một từ điển thay vì một tuple: `{"num": X_train_num, "cat": X_train_cat}`. Ngoài ra, chúng ta có thể đã truyền một `tf.data.Dataset` chứa các batch, mỗi batch được biểu diễn dưới dạng `((X_batch_num, X_batch_cat), y_batch)` hoặc dưới dạng `({"num": X_batch_num, "cat": X_batch_cat}, y_batch)`. Và tất nhiên điều tương tự cũng áp dụng cho dữ liệu xác thực.

OK, bây giờ bạn đã học cách mã hóa các đặc trưng phân loại, đã đến lúc chúng ta chuyển sự chú ý sang tiền xử lý văn bản.

Tiền xử lý văn bản

Keras cung cấp một lớp `TextVectorization` để tiền xử lý văn bản cơ bản. Giống như lớp `StringLookup`, bạn phải truyền cho nó một từ vựng khi tạo, hoặc để nó học từ vựng từ một số dữ liệu huấn luyện bằng phương thức `adapt()`. Hãy xem một ví dụ:

```

>>> train_data = ["To be", "!(to be)", "That's the question", "Be, be, be."]

>>> text_vec_layer = tf.keras.layers.TextVectorization()

>>> text_vec_layer.adapt(train_data)

>>> text_vec_layer(["Be good!", "Question: be or be?"])
<tf.Tensor:

```



```
shape=(2, 4), dtype=int64, numpy= array([[2, 1, 0, 0],
      [6, 2, 1, 2]])>
```

Hai câu “Be good!” và “Question: be or be?” được mã hóa lần lượt là [2, 1, 0, 0] và [6, 2, 1, 2]. Từ vựng được học từ bốn câu trong dữ liệu huấn luyện: “be” = 2, “to” = 3, v.v. Để xây dựng từ vựng, phương thức `adapt()` trước tiên chuyển các câu huấn luyện sang chữ thường và loại bỏ dấu câu, đó là lý do tại sao “Be”, “be”, và “be?” đều được mã hóa là “be” = 2. Tiếp theo, các câu được tách trên khoảng trắng, và các từ kết quả được sắp xếp theo tần suất giảm dần, tạo ra từ vựng cuối cùng. Khi mã hóa câu, các từ không xác định được mã hóa là 1. Cuối cùng, vì câu đầu tiên ngắn hơn câu thứ hai, nó được đệm bằng 0.

Các ID từ phải được mã hóa, thường sử dụng lớp `Embedding`: chúng ta sẽ làm điều này trong Chương 16.

Ngoài ra, bạn có thể đặt đối số `output_mode` của lớp `TextVectorization` thành “multi_hot” hoặc “count” để nhận được các mã hóa tương ứng. Tuy nhiên, việc chỉ đếm từ thường không lý tưởng: các từ như “to” và “the” quá thường xuyên đến mức chúng hầu như không quan trọng chút nào, trong khi các từ hiếm hơn như “basketball” lại mang nhiều thông tin hơn. Vì vậy, thay vì đặt `output_mode` thành “multi_hot” hoặc “count”, thường nên đặt nó thành “tf_idf”, viết tắt của “term-frequency $\times$ inverse-document-frequency” (TF-IDF). Điều này tương tự như mã hóa đếm, nhưng các từ xuất hiện thường xuyên trong dữ liệu huấn luyện bị giảm trọng số, và ngược lại, các từ hiếm bị tăng trọng số. Ví dụ:

```
>>> text_vec_layer = tf.keras.layers.TextVectorization(output_mode="tf_idf")

>>> text_vec_layer.adapt(train_data)

>>> text_vec_layer(["Be good!", "Question: be or be?"])
<tf.Tensor:
shape=(2, 6), dtype=float32, numpy= array([[0.96725637, 0.6931472 , 0.
, 0.
, 0.
, 0.
],
      [0.96725637, 1.3862944 , 0.
, 0.
, 0.
, 0.
], dtype=float32)>
```

Có nhiều biến thể TF-IDF, nhưng cách lớp `TextVectorization` triển khai nó là bằng cách nhân mỗi số đếm từ với một trọng số bằng $\log(1 + d/(f + 1))$, trong đó d là tổng số câu (còn gọi là tài liệu) trong dữ liệu huấn luyện và f đếm số câu huấn luyện chứa từ đã cho. Ví dụ, trong trường hợp này có $d = 4$ câu trong dữ liệu huấn luyện, và từ “be” xuất hiện trong $f = 3$ trong số đó. Vì từ “be” xuất hiện hai lần trong câu “Question: be or be?”, nó được mã hóa là $2 \times \log(1 + 4/(1 + 3)) \approx 1.3862944$. Từ “question” chỉ xuất hiện một lần, nhưng vì nó là một từ ít phổ biến hơn, mã hóa của nó gần như cao: $1 \times \log(1 + 4/(1 + 1)) \approx 1.0986123$. Lưu ý rằng trọng số trung bình được sử dụng cho các từ không xác định.

Cách tiếp cận mã hóa văn bản này dễ sử dụng và nó có thể mang lại kết quả khá tốt cho các tác vụ xử lý ngôn ngữ tự nhiên cơ bản, nhưng nó có một số hạn chế quan trọng: nó chỉ hoạt

động với các ngôn ngữ tách từ bằng khoảng trắng, nó không phân biệt giữa các từ đồng âm (ví dụ: “to bear” so với “teddy bear”), nó không gợi ý cho mô hình của bạn rằng các từ như “evolution” và “evolutionary” có liên quan, v.v. Và nếu bạn sử dụng mã hóa multi-hot, count hoặc TF-IDF, thì thứ tự của các từ bị mất. Vậy các lựa chọn khác là gì?

Một lựa chọn là sử dụng thư viện TensorFlow Text, cái này cung cấp các tính năng tiền xử lý văn bản nâng cao hơn lớp TextVectorization. Ví dụ, nó bao gồm một số bộ tách từ subword có khả năng chia văn bản thành các token nhỏ hơn từ, điều này giúp mô hình dễ dàng phát hiện rằng “evolution” và “evolutionary” có điểm chung (thêm về tokenization subword trong Chương 16).

Một lựa chọn khác là sử dụng các thành phần mô hình ngôn ngữ đã được huấn luyện trước. Hãy xem xét điều này ngay bây giờ.

Sử dụng các thành phần mô hình ngôn ngữ được huấn luyện trước

Thư viện TensorFlow Hub giúp dễ dàng tái sử dụng các thành phần mô hình đã được huấn luyện trước trong các mô hình của riêng bạn, cho văn bản, hình ảnh, âm thanh và hơn thế nữa. Các thành phần mô hình này được gọi là **module**. Chỉ cần duyệt kho lưu trữ TF Hub, tìm cái bạn cần và sao chép ví dụ mã vào dự án của bạn, và module sẽ tự động được tải xuống và đóng gói thành một lớp Keras mà bạn có thể trực tiếp đưa vào mô hình của mình. Các module thường chứa cả mã tiền xử lý và trọng số đã được huấn luyện trước, và chúng thường không yêu cầu huấn luyện thêm (nhưng tất nhiên, phần còn lại của mô hình của bạn chắc chắn sẽ yêu cầu huấn luyện).

Ví dụ, một số mô hình ngôn ngữ mạnh mẽ đã được huấn luyện trước có sẵn. Những mô hình mạnh mẽ nhất khá lớn (vài gigabyte), vì vậy để có một ví dụ nhanh, hãy sử dụng module nnlm-en-dim50, phiên bản 2, đây là một module khá cơ bản nhận văn bản thô làm đầu vào và xuất ra các embedding câu 50 chiều. Chúng ta sẽ nhập TensorFlow Hub và sử dụng nó để tải module, sau đó sử dụng module đó để mã hóa hai câu thành vector:

```
>>> import tensorflow_hub as hub

>>> hub_layer = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim50/2")

>>> sentence_embeddings = hub_layer(tf.constant(["To be", "Not to be"]))

>>> sentence_embeddings.numpy().round(2)
array([[ -0.25,  0.28,  0.01,  0.1 , ...,  0.05,  0.31],
       [ -0.2 ,  0.2 , -0.08,  0.02, ..., -0.04,  0.15]],
      dtype=float32)
```

Lớp `hub.KerasLayer` tải module từ URL đã cho. Module cụ thể này là một bộ mã hóa câu: nó nhận các chuỗi làm đầu vào và mã hóa mỗi chuỗi thành một vector duy nhất (trong trường hợp này, một vector 50 chiều). Bên trong, nó phân tích cú pháp chuỗi (tách từ trên khoảng trắng) và nhúng từng từ bằng cách sử dụng một ma trận embedding đã được huấn luyện

trước trên một tập hợp lớn: tập hợp Google News 7B (dài bảy tỷ từ!). Sau đó, nó tính toán trung bình của tất cả các embedding từ, và kết quả là embedding câu.

Bạn chỉ cần đưa `hub_layer` này vào mô hình của mình, và bạn đã sẵn sàng. Lưu ý rằng mô hình ngôn ngữ cụ thể này được huấn luyện trên ngôn ngữ tiếng Anh, nhưng nhiều ngôn ngữ khác cũng có sẵn, cũng như các mô hình đa ngôn ngữ.

Cuối cùng nhưng không kém phần quan trọng, thư viện Transformers mã nguồn mở tuyệt vời của Hugging Face cũng giúp dễ dàng đưa các thành phần mô hình ngôn ngữ mạnh mẽ vào các mô hình của riêng bạn. Bạn có thể duyệt Hugging Face Hub, chọn mô hình bạn muốn và sử dụng các ví dụ mã được cung cấp để bắt đầu. Nó từng chỉ chứa các mô hình ngôn ngữ, nhưng hiện đã mở rộng để bao gồm các mô hình hình ảnh và hơn thế nữa.

Chúng ta sẽ quay lại xử lý ngôn ngữ tự nhiên sâu hơn trong Chương 16. Bây giờ hãy xem các lớp tiền xử lý hình ảnh của Keras.

Các lớp tiền xử lý hình ảnh

API tiền xử lý của Keras bao gồm ba lớp tiền xử lý hình ảnh:

- `tf.keras.layers.Resizing` thay đổi kích thước hình ảnh đầu vào theo kích thước mong muốn. Ví dụ, `Resizing(height=100, width=200)` thay đổi kích thước mỗi hình ảnh thành 100×200 , có thể làm biến dạng hình ảnh. Nếu bạn đặt `crop_to_aspect_ratio=True`, thì hình ảnh sẽ được cắt theo tỷ lệ hình ảnh mục tiêu, để tránh biến dạng.
- `tf.keras.layers.Rescaling` điều chỉnh lại các giá trị pixel. Ví dụ, `Rescaling(scale=2/255, offset=-1)` điều chỉnh lại các giá trị từ $0 \rightarrow 255$ thành $-1 \rightarrow 1$.
- `tf.keras.layers.CenterCrop` cắt hình ảnh, chỉ giữ lại một phần trung tâm có chiều cao và chiều rộng mong muốn.

Ví dụ, hãy tải một vài hình ảnh mẫu và cắt chúng ở giữa. Để làm điều này, chúng ta sẽ sử dụng hàm `load_sample_images()` của Scikit-Learn; hàm này tải hai hình ảnh màu, một là đèn thờ Trung Quốc và một là hoa (điều này yêu cầu thư viện Pillow, cái này đã được cài đặt nếu bạn đang sử dụng Colab hoặc nếu bạn đã làm theo hướng dẫn cài đặt):

```
from sklearn.datasets import load_sample_images

images = load_sample_images()["images"]
crop_image_layer = tf.keras.layers.CenterCrop(height=100, width=100)
cropped_images = crop_image_layer(images)
```

Keras cũng bao gồm một số lớp để tăng cường dữ liệu, chẳng hạn như `RandomCrop`, `RandomFlip`, `RandomTranslation`, `RandomRotation`, `RandomZoom`, `RandomHeight`, `RandomWidth`, và `RandomContrast`. Các lớp này chỉ hoạt động trong quá trình huấn luyện, và chúng ngẫu nhiên áp dụng một số phép biến đổi cho hình ảnh đầu vào (tên của chúng tự giải thích). Tăng cường dữ liệu sẽ làm tăng kích thước của tập huấn luyện một cách nhân tạo,

điều này thường dẫn đến cải thiện hiệu suất, miễn là các hình ảnh đã biến đổi trông giống như hình ảnh thực tế (không tăng cường). Chúng ta sẽ xem xét kỹ hơn quá trình xử lý hình ảnh trong chương tiếp theo.

Bây giờ hãy xem một cách khác để tải dữ liệu dễ dàng và hiệu quả trong TensorFlow.

Dự án TensorFlow Datasets

Dự án TensorFlow Datasets (TFDS) giúp rất dễ dàng tải các tập dữ liệu phổ biến, từ nhỏ như MNIST hoặc Fashion MNIST đến các tập dữ liệu lớn như ImageNet (bạn sẽ cần khá nhiều không gian đĩa!). Danh sách bao gồm các tập dữ liệu hình ảnh, tập dữ liệu văn bản (bao gồm các tập dữ liệu dịch thuật), tập dữ liệu âm thanh và video, chuỗi thời gian, và nhiều hơn nữa. Bạn có thể truy cập <https://homl.info/tfds> để xem danh sách đầy đủ, cùng với mô tả của mỗi tập dữ liệu. Bạn cũng có thể kiểm tra KnowYourData, đây là một công cụ để khám phá và hiểu nhiều tập dữ liệu được TFDS cung cấp.

TFDS không được gói cùng với TensorFlow, nhưng nếu bạn đang chạy trên Colab hoặc nếu bạn đã làm theo hướng dẫn cài đặt tại <https://homl.info/install>, thì nó đã được cài đặt. Sau đó, bạn có thể nhập `tensorflow_datasets`, thường là dưới dạng `tfds`, sau đó gọi hàm `tfds.load()`, hàm này sẽ tải dữ liệu bạn muốn (trừ khi nó đã được tải trước đó) và trả về dữ liệu dưới dạng một từ điển các tập dữ liệu (thường là một cho huấn luyện và một cho kiểm tra, nhưng điều này tùy thuộc vào tập dữ liệu bạn chọn). Ví dụ, hãy tải MNIST:

```
import tensorflow_datasets as tfds

datasets = tfds.load(name="mnist")
mnist_train, mnist_test = datasets["train"], datasets["test"]
```

Sau đó, bạn có thể áp dụng bất kỳ phép biến đổi nào bạn muốn (thường là xáo trộn, nhóm batch và prefetching), và bạn đã sẵn sàng để huấn luyện mô hình của mình.

Dưới đây là một ví dụ đơn giản:

```
for batch in mnist_train.shuffle(10_000, seed=42).batch(32).prefetch(1):
    images = batch["image"]
    labels = batch["label"]
    # ... làm điều gì đó với hình ảnh và nhãn
```

Lưu ý rằng mỗi mục trong tập dữ liệu là một từ điển chứa cả đặc trưng và nhãn. Nhưng Keras mong đợi mỗi mục là một tuple chứa hai phần tử (một lần nữa, các đặc trưng và nhãn). Bạn có thể biến đổi tập dữ liệu bằng phương thức `map()`, như thế này:

```
mnist_train = mnist_train.shuffle(buffer_size=10_000, seed=42).batch(32)

mnist_train = mnist_train.map(lambda items:
                               (items["image"], items["label"]))
mnist_train = mnist_train.prefetch(1)
```

Nhưng đơn giản hơn là yêu cầu hàm `load()` làm điều này cho bạn bằng cách đặt `as_supervised=True` (rõ ràng điều này chỉ hoạt động đối với các tập dữ liệu được gắn nhãn).

Cuối cùng, TFDS cung cấp một cách tiện lợi để chia dữ liệu bằng đối số `split`. Ví dụ, nếu bạn muốn sử dụng 90% đầu tiên của tập huấn luyện để huấn luyện, 10% còn lại để xác thực, và toàn bộ tập kiểm tra để kiểm tra, thì bạn có thể đặt `split=["train[:90%]", "train[90%:]", "test"]`. Hàm `load()` sẽ trả về cả ba tập hợp. Dưới đây là một ví dụ hoàn chỉnh, tải và chia tập dữ liệu MNIST bằng TFDS, sau đó sử dụng các tập hợp này để huấn luyện và đánh giá một mô hình Keras đơn giản:

```
train_set, valid_set, test_set = tfds.load(
    name="mnist",
    split=["train[:90%]", "train[90%:]", "test"],
    as_supervised=True
)
train_set = train_set.shuffle(buffer_size=10_000,
                             seed=42).batch(32).prefetch(1)
valid_set = valid_set.batch(32).cache()
test_set = test_set.batch(32).cache()
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=5)
test_loss, test_accuracy = model.evaluate(test_set)
```

Chúc mừng, bạn đã đến cuối chương khá kỹ thuật này! Bạn có thể cảm thấy rằng nó hơi xa vời so với vẻ đẹp trừu tượng của mạng nơ-ron, nhưng thực tế là học sâu thường liên quan đến lượng lớn dữ liệu, và biết cách tải, phân tích cú pháp và tiền xử lý nó một cách hiệu quả là một kỹ năng rất quan trọng. Trong chương tiếp theo, chúng ta sẽ xem xét mạng nơ-ron tích chập, đây là một trong những kiến trúc mạng nơ-ron thành công nhất cho xử lý hình ảnh và nhiều ứng dụng khác.

Bài tập

1. Tại sao bạn muốn sử dụng API `tf.data`?
2. Lợi ích của việc chia một tập dữ liệu lớn thành nhiều tệp là gì?
3. Trong quá trình huấn luyện, làm thế nào bạn có thể biết rằng pipeline đầu vào của bạn là nút thắt cổ chai? Bạn có thể làm gì để khắc phục nó?
4. Bạn có thể lưu bất kỳ dữ liệu nhị phân nào vào tệp TFRecord, hay chỉ các bộ đệm giao thức đã được tuần tự hóa?
5. Tại sao bạn lại phải mất công chuyển đổi tất cả dữ liệu của mình sang định dạng `protobuf Example`? Tại sao không sử dụng định nghĩa `protobuf` của riêng bạn?

6. Khi sử dụng TFRecord, khi nào bạn muốn kích hoạt nén? Tại sao không làm điều đó một cách hệ thống?
7. Dữ liệu có thể được tiền xử lý trực tiếp khi ghi các tệp dữ liệu, hoặc trong pipeline `tf.data`, hoặc trong các lớp tiền xử lý trong mô hình của bạn. Bạn có thể liệt kê một vài ưu và nhược điểm của mỗi tùy chọn không?
8. Kể tên một vài cách phổ biến bạn có thể mã hóa các đặc trưng số nguyên phân loại. Còn văn bản thì sao?
9. Tải tập dữ liệu Fashion MNIST (được giới thiệu trong Chương 10); chia nó thành một tập huấn luyện, một tập xác thực và một tập kiểm tra; xáo trộn tập huấn luyện; và lưu mỗi tập dữ liệu vào nhiều tệp TFRecord. Mỗi bản ghi nên là một protobuf Example đã được tuần tự hóa với hai đặc trưng: hình ảnh đã được tuần tự hóa (sử dụng `tf.io.serialize_tensor()` để tuần tự hóa mỗi hình ảnh), và nhãn. Sau đó, sử dụng `tf.data` để tạo một tập dữ liệu hiệu quả cho mỗi tập hợp. Cuối cùng, sử dụng một mô hình Keras để huấn luyện các tập dữ liệu này, bao gồm một lớp tiền xử lý để chuẩn hóa mỗi đặc trưng đầu vào. Cố gắng làm cho pipeline đầu vào hiệu quả nhất có thể, sử dụng TensorBoard để trực quan hóa dữ liệu lập hồ sơ.
10. Trong bài tập này, bạn sẽ tải xuống một tập dữ liệu, chia nó, tạo một `tf.data.Dataset` để tải và tiền xử lý nó một cách hiệu quả, sau đó xây dựng và huấn luyện một mô hình phân loại nhị phân chứa một lớp Embedding:
 - a. Tải xuống Large Movie Review Dataset, cái này chứa 50.000 bài đánh giá phim từ Internet Movie Database (IMDb). Dữ liệu được tổ chức trong hai thư mục, `train` và `test`, mỗi thư mục chứa một thư mục con `pos` với 12.500 bài đánh giá tích cực và một thư mục con `neg` với 12.500 bài đánh giá tiêu cực. Mỗi bài đánh giá được lưu trữ trong một tệp văn bản riêng biệt. Có các tệp và thư mục khác (bao gồm các phiên bản bag-of-words đã được tiền xử lý), nhưng chúng ta sẽ bỏ qua chúng trong bài tập này.
 - b. Chia tập kiểm tra thành một tập xác thực (15.000) và một tập kiểm tra (10.000).
 - c. Sử dụng `tf.data` để tạo một tập dữ liệu hiệu quả cho mỗi tập hợp.
 - d. Tạo một mô hình phân loại nhị phân, sử dụng lớp `TextVectorization` để tiền xử lý mỗi bài đánh giá.
 - e. Thêm một lớp Embedding và tính toán embedding trung bình cho mỗi bài đánh giá, nhân với căn bậc hai của số từ (xem Chương 16). Embedding trung bình đã được điều chỉnh tỷ lệ này sau đó có thể được truyền cho phần còn lại của mô hình của bạn.
 - f. Huấn luyện mô hình và xem độ chính xác bạn đạt được. Cố gắng tối ưu hóa các pipeline của bạn để quá trình huấn luyện nhanh nhất có thể.
 - g. Sử dụng TFDS để tải cùng tập dữ liệu dễ dàng hơn: `tfds.load("imdb_reviews")`.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.